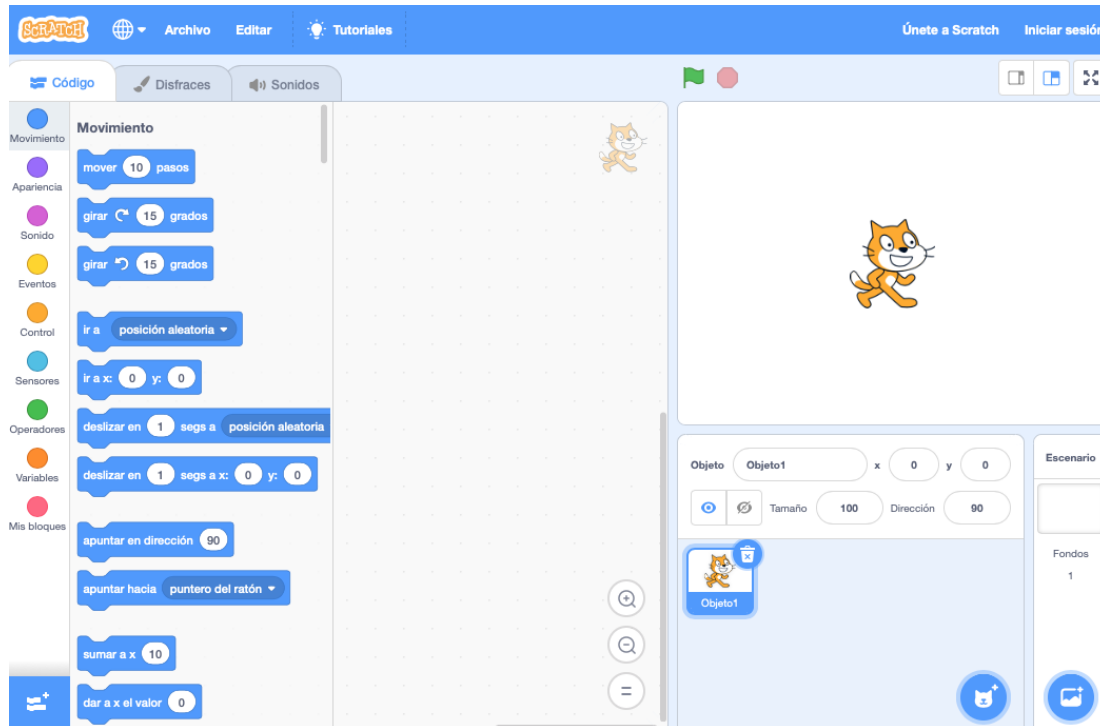


SCRATCH

Programación



ÍNDICE

PRIMEROS PASOS	3
VARIABLES Y ENTRADA/SALIDA DE DATOS.....	4
VARIABLES PROPIAS DE SCRATCH	4
DECLARACIÓN DE VARIABLES	4
INTRODUCCIÓN DE DATOS Y OPERADORES.....	5
CONDICIONALES Y CONDICIONES.....	6
CONDICIONALES	6
CONDICIONES.....	7
ITERATIVAS	8
<i>Juego con “Repite”</i>	<i>8</i>
<i>Juego con “Repite Hasta”</i>	<i>9</i>

PRIMEROS PASOS

Se puede acceder a la página de Scratch desde el siguiente enlace: <https://scratch.mit.edu>

En esta página podrás comenzar tus proyectos directamente desde el enlace “Crear” o descargar la versión de Escritorio desde *Recursos* → *Descargar*.

El funcionamiento de Scratch es muy sencillo. El objetivo de iniciar la asignatura con Scratch es la visualización de conceptos de programación.

Nada más abrir Scratch hay tres zonas, una con un gato (arriba una bandera verde y un octógono (señal de Stop)), otra con bloques de color en unas pestañas y una zona vacía.

Por ahora sólo vamos a usar bloques con muescas tanto arriba como abajo.

Arriba, pulsa la pestaña Código y elige Movimiento.

Arrastra el elemento o bloque “mover 10 pasos” a la zona de la derecha y haz doble clic sobre él. El gato se mueve hacia la derecha.

Escribe el número 100 en vez del 10 que está en el bloque y haz doble clic sobre el bloque.

El gato se mueve más hacia la derecha. Puedes arrastrar al gato hacia el centro.

Vamos a añadir un giro, arrastra el bloque “gira 15 grados” y sustituye por 90. Elige hacia dónde debe girar. Encaja un bloque con otro. Verás que una zona se pone de color blanco, en ese momento suelta el bloque y se unirán. Haz clic sobre uno de los dos bloques y se realizará la acción. El resultado es el de la imagen. Si haces clic cuatro veces seguidas, el gato se moverá haciendo un cuadrado.



VARIABLES Y ENTRADA/SALIDA DE DATOS

Vamos a ver qué es una variable y cómo se pueden usar. También veremos cómo se introducen y muestran datos en Scratch.

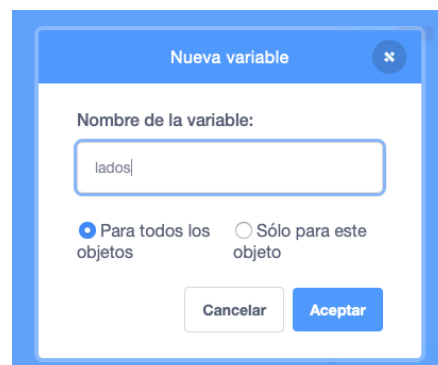
Variables propias de Scratch

Una variable es un lugar en la memoria del ordenador donde se almacena un tipo de dato. Según el lenguaje de programación se puede diferenciar entre diferentes tipos de datos, como números enteros, números reales, caracteres (que incluyen letras, cifras y símbolos del teclado), cadenas de caracteres (textos) y combinaciones de los anteriores. Cada lenguaje de programación pone los límites en cómo se mezclan o distinguen los tipos de datos, el número máximo que se puede almacenar, y otros detalles en los que no nos vamos a fijar ahora.

Declaración de variables

Con las variables propias del Scratch (color, coordenadas y alguna más que no nos importa mucho) no se pueden hacer muchas cosas. Por ejemplo, si queremos contar los lados del polígono que llevamos, necesitamos una variable que contenga ese valor. Para ello:

1. Pulsa Variables
2. Pulsa Crear una variable
3. Teclea “lados” como nombre de la variable.



Prepara un programa, que cuenta los lados hechos.



La primera instrucción (bloque dar a) se llama inicialización, nada más empezar da un valor inicial a la variable lados con 0.

En sumar a lados le suma 1 al valor que tenía, la primera vez hará $0+1 = 1$, la segunda $1+1 = 2$ etc.

La instrucción decir muestra el contenido, el valor guardado en la variable lados en la pantalla, como un bocadillo del gato.

Introducción de datos y operadores

Para introducir datos al programa (lo que se llama en informática entrada/salida) usamos la instrucción o bloque “preguntar y esperar” junto con la variable “respuesta”.

También hay operadores, en este caso la multiplicación.

Prueba qué hace un programa en el que tengas un valor en una variable “cantidad” y ponga un bloque parecido al anterior: dar a cantidad el valor $\text{cantidad} * 2$



CONDICIONALES Y CONDICIONES

Condicionales

Vamos a hacer que el dodecágono tenga lados de colores alternados, es decir, que los lados pares sean de un color y los impares de otro. Para ello, vamos a usar la variable que numeraba los lados del último programa.

Pero primero vamos a ver cómo detectamos los pares. Queremos algo parecido a:

Si "lados" es par entonces di ¡Par!

¿Pero cómo sabemos si el número es par? Hay una operación que consiste en quedarnos con el resto de la división entera, y se llama módulo. Por eso, 7 módulo 2 nos da 1 y 6 módulo 2 nos da 0. Es un poco laborioso crear la instrucción-bloque compuesta de bloques, pero es fácil. Debe quedar aproximadamente como en la imagen.

Y si queremos que diga impar, usamos la estructura condicional con forma de E si-entonces-aino. En la parte de abajo pondremos que diga Impar. El si quedaría así:



Condiciones

Las condiciones tienen forma de bloque de extremos en ángulo recto. Por una parte tenemos los comparadores $>$; $<$; $=$ que se pueden usar entre dos números o bloques de extremos redondeados.

En los cuadrados se pueden escribir números o cadenas de caracteres (textos), además de bloques redondeados o “angulosos”.

Por otra parte, tenemos las operaciones booleanas: conjunción, disyunción y negación, que se escriben y , o , no , respectivamente. Y necesita que se cumplan las dos condiciones, O una por lo menos, y NO hace lo contrario.

$3 < 5$ y $7 < 4$ es falso

$3 < 5$ o $7 < 4$ es verdadero

$no(7 < 4)$ es verdadero

Una comparación da verdadero o falso (true, false). Si estamos comparando cadenas (texto) se usa el alfabeto siendo menor a que b y aa que ab , pero mayor ba que ab .

La respuesta “true” o “false” se da a su vez en forma de cadena. Por ello hay que tener cuidado al comparar tres números. Para ver si $3 < x < 5$ tendremos que construir

$3 < x$ y $x < 5$

o de forma equivalente,

$(x > 3)$ y $(x < 5)$

También tendremos que usar y para $>$ o $<$ y no para $=$. Ante todo, hay que pensar qué se quiere hacer y qué equivalencias entre operaciones se pueden aplicar para simplificar la expresión.

De la misma forma que matemáticas se define el orden de evaluación o precedencia con paréntesis y corchetes, en Scratch se define con el orden en el que se introducen unos bloques en otros. Se empieza a evaluar por el más interno y se acaba por el más externo.

Los bloques redondeados en el apartado “Operadores” son operaciones matemáticas y otros elementos, y producen números y textos, es decir, pueden sustituir a cualquier valor constante (como los grados que hay que girar o lo que tiene que pensar el gatete) y pueden usarse también como si fueran una variable. La ventaja del Scratch en este punto es clara, puesto que sólo deja usar un bloque donde tiene sentido.

ITERATIVAS

Con un bloque de control iterativo podemos ejecutar un subprograma un número determinado de veces. Como ejemplo, podemos preparar un juego como el que se presenta a continuación.

Juego con “Repite”

Con lo que sabemos, casi se puede hacer el juego de la imagen. No voy a explicar qué hace en detalle cada parte, porque ése es tu objetivo, copiarlo, probarlo y deducir qué hace cada bloque.

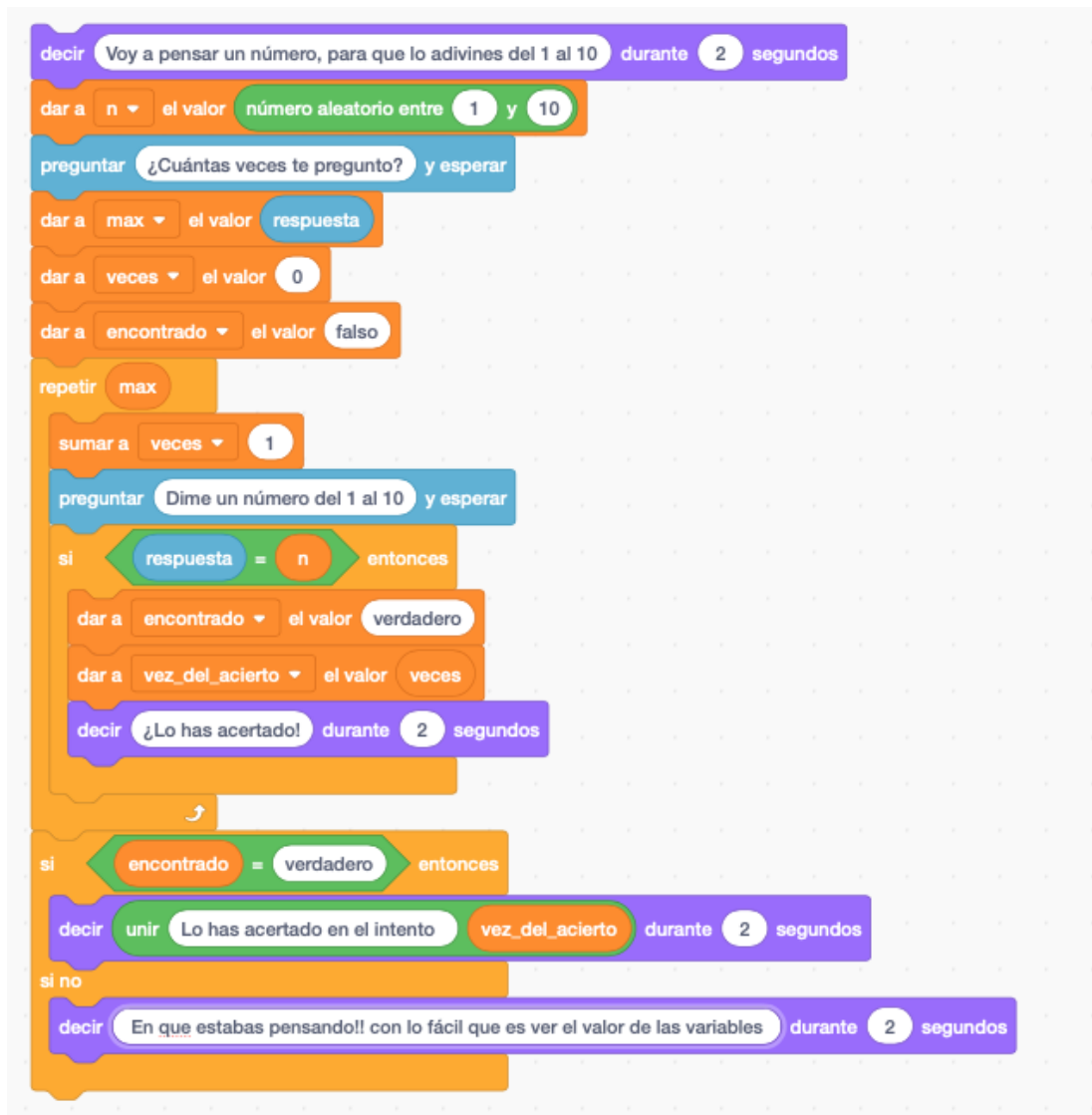
Lo único que nos falta para entenderlo es saber dos cosas. La primera es cómo hacer que nos tecleen algo, ya sea un número o una cadena, para entender el juego. Vemos los bloques azules, “Pregunta y espera”. Lo que hace es que el gato presenta el texto incluido y se para hasta que tecleamos algo y damos al tick de aceptar. Lo tecleado se almacena en la variable especial “Respuesta”.

Hay otro bloque que no habíamos usado, y es “unir”. Lo que hace es concatenar, la operación de pegar un texto con otro, y es muy útil para que aparezca un texto antes de una variable. En este lenguaje de programación se convierte automáticamente el contenido de la variable a texto para concatenarlo.

Es muy importante que entiendas bien cada uno de los pasos que da el programa de la imagen antes de continuar, puesto que de lo contrario no se puede entender nada de lo siguiente.

Un truco: pártelo en trozos y ejecútalos de uno en uno, haciendo clic en cada uno y viendo mientras qué valor reciben las variables.

He colocado algún comentario como pista, pero es necesario ejecutarlo en el ordenador para comprenderlo. Precisamente es una buena práctica la de decir qué significa cada variable, qué sentido tiene el valor que contiene, si es la cantidad de veces que se hace algo, si es el nombre de un jugador, si sirve para saber si se ha encontrado o no la respuesta ...



Juego con “Repite Hasta”

De todas formas, el juego que se ha programado en la imagen anterior es un poco extraño, porque se suele acabar de jugar cuando se acierta el número. En este caso el programa sigue preguntando cuando se ha acertado. Para ello vamos a usar una nueva estructura iterativa que repite hasta que se cumple una cierta condición. La instrucción es “repite hasta” <condición> . En nuestro caso, la condición que se debe cumplir para dejar de preguntar es que se haya acertado el número pensado por el gato (almacenado en la variable n).

Vamos a centrarnos en la parte del programa que se repite un número determinado de veces con la instrucción repite. Queremos que se repita un número indeterminado de repeticiones con el repite hasta. Es importante distinguir el número de veces que se necesita repetir para solucionar el problema. En este caso quedaría así:

Repite hasta que encontrado=verdadero

Cambia veces en 1

Pide "Dime un número del 1 al 10" y espera

Si respuesta=n entonces

Asigna a encontrado el valor verdadero

Asigna a vez_del_acierto el valor de veces

Observa que la condicional posterior a este trozo, que compara encontrado con verdadero, siempre se cumplirá puesto que sólo se sale del bucle o ciclo repetitivo cuando se produce esta situación, que el número haya sido acertado.

Ahora vamos a restringir el número de veces que se pregunta al valor que nos dan al preguntar, y que introducimos en la variable max.

Repite hasta que encontrado=verdadero o veces=max

Cambia veces en 1

Pide "Dime un número del 1 al 10" y espera

Si respuesta=n entonces

Asigna a encontrado el valor verdadero

Asigna a vez_del_acierto el valor de veces

En este caso es necesaria la condición que explica al usuario si se ha salido del bucle porque se ha acertado el número o porque se ha agotado el número de intentos. Además pueden ocurrir las dos razones a la vez, ya que se puede adivinar el número en el último intento válido, con lo que la siguiente vuelta se tendría que salir por ambas situaciones.

Con este análisis llegamos a la conclusión de que hace falta primero entender el problema y después formalizarlo con un programa. Lo anterior lo podríamos simplificar así:

Pide "Dime un número del 1 al 10" y espera

Repite hasta que respuesta=n o veces=max

Cambia veces en 1

Pide "Dime un número del 1 al 10" y espera

Asigna a vez_del_acierto el valor de veces

La primera vez que se pide es una instrucción que se ejecuta una sola vez y prepara la situación para que se pueda hacer la primera comparación al entrar en el bucle. En este caso (esta instrucción y con este lenguaje de programación), la condición se comprueba antes de empezar la primera instrucción de cada vuelta del ciclo.

